1. Lab 13: Transport Layer Security (TLS), PKI, and Routed DNS Infrastructure Setup

Lab Name: Lab 13: Transport Layer Security (TLS), PKI, and Routed DNS Infrastructure Setup **Date:** YYYY-MM-DD **Name:** [Your Name] **Lab Partners:** [Partners' Names, if any] **Software/Hardware Used:** Cisco Modeling Labs (CML), Cisco IOSv / Linux Router, OpenSSL, `dnsmasq`, Nginx, `curl`, Wireshark

---

2. Background

This capstone lab brings together a comprehensive synthesis of the core networking and security principles you have mastered throughout the semester. In previous labs, you explored isolated protocol layers—such as IP addressing and subnetting, inter-subnet routing, DNS domain resolution, TCP connection management, and link-layer bridging. In this lab, you will combine all of these elements into a single, fully integrated end-to-end secure web infrastructure.

Modern secure web communications rely on the seamless integration of four fundamental network technologies:

1. **Inter-Subnet IP Routing:** Routers forward IP packets across distinct network subnets (e.g., Client LAN to Server DMZ) using default gateways and routing tables.
2. **Domain Name System (DNS):** Translates human-readable domain names (e.g., `<lastname>.cs457.lab`) into IP addresses (`10.1.2.100`) over UDP Port 53.
3. **Public Key Infrastructure (PKI):** Establishes digital identity and trust. A **Root Certification Authority (CA)** signs digital certificates (`server.crt`), binding a web server's identity to its public key.
4. **Transport Layer Security (TLS):** Operating over TCP Port 443, TLS uses asymmetric cryptography for server authentication and session key exchange, followed by symmetric key encryption (e.g., AES) for bulk application data (HTTPS).

When a user visits an HTTPS web page:

- The client queries its configured **DNS Server** to resolve the server's hostname to an IP address.
- The client establishes a **TCP 3-Way Handshake** across intermediate **Routers** to the destination web server.
- The client performs a **TLS Handshake**, receiving the server's X.509 certificate.
- If the client trusts the **Root CA** that signed the certificate, the client validates the identity without warnings and establishes an encrypted HTTPS session.

In this lab, you will construct and configure a multi-subnet CML topology featuring an Edge Router, a DNS Server, a Web Server running a Private PKI, and a Web Client, analyzing the complete multi-layer protocol stack in Wireshark.

---

3. Objective / Purpose

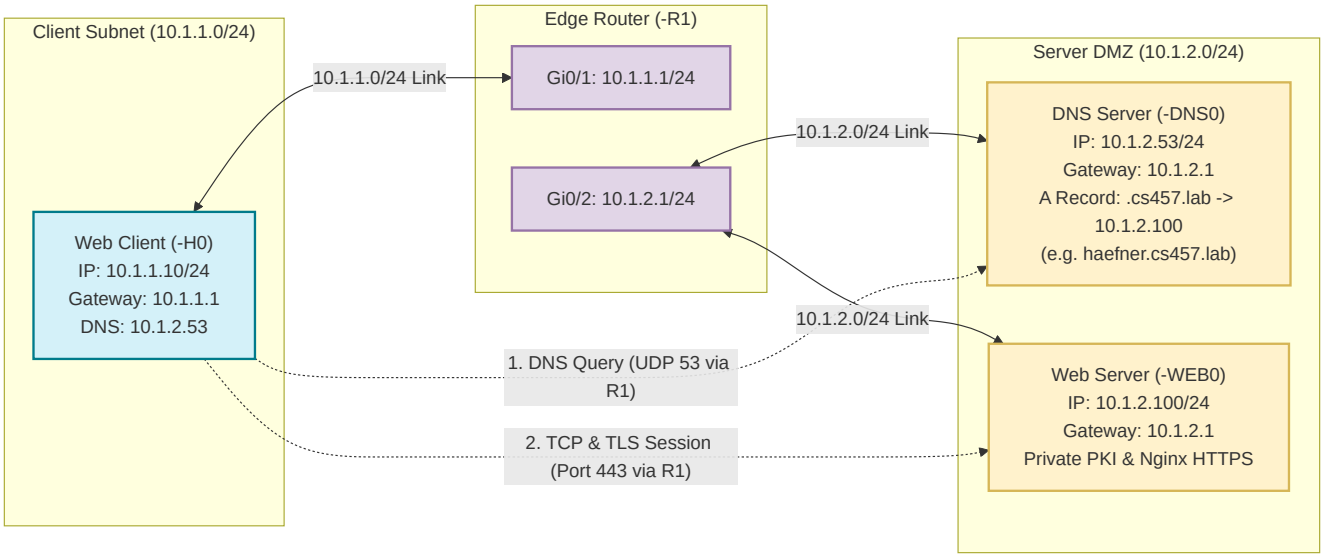
To configure an inter-subnet router, set up a DNS server for custom domain resolution, build a private PKI to issue X.509 certificates, configure an HTTPS web server, push the Root CA certificate to the client, and capture/analyze the complete DNS, TCP, and TLS protocol flow in Wireshark.

---

4. Topology Diagram

> *[!IMPORTANT]* ***Custom Student Domain Naming Rule:*** *Every student* ***MUST*** *use their own lower-case last name for the domain name:* `<lastname>.cs457.lab`.
>
> - ***Example (Prof. Haefner):*** `haefner.cs457.lab`

> - *Replace `<lastname>` with your actual last name in all DNS configurations, OpenSSL certificate requests, Nginx server blocks, `nslookup`, and `curl` tests!*



Details:

- Client Subnet: `10.1.1.0/24` (Gateway: `10.1.1.1`)
- Server Subnet: `10.1.2.0/24` (Gateway: `10.1.2.1`)
- Router (`<initials>-R1`): Dual-homed routing between Client LAN and Server DMZ
- Services: DNS (`10.1.2.53`), HTTPS (`10.1.2.100` FQDN: `<lastname>.cs457.lab`, e.g. `haefner.cs457.lab`)

---

5. Equipment & Environment

- **Software:** Cisco Modeling Labs (CML), Cisco IOSv / Linux Router, OpenSSL, `dnsmasq`, Nginx, `curl`, Wireshark.
- **Nodes:** 1x Edge Router (`<initials>-R1`), 1x Linux Client (`<initials>-H0`), 1x Linux DNS Server (`<initials>-DNS0`), 1x Linux Web Server (`<initials>-WEB0`).

---

6. Step-by-Step Configuration Guide

> *[!IMPORTANT] **Domain Name Customization:** Throughout these steps, replace `<lastname>` with your lower-case last name (e.g., `haefner.cs457.lab`).*

Step 1: Configure Inter-Subnet Router (`<initials>-R1`)

Open the console of Router **`<initials>-R1`** and configure interface IP addresses and IP routing:

- **Cisco IOSv Router Configuration:**

```
enable
configure terminal
hostname KH-R1

! Client Subnet Interface
```

```
interface GigabitEthernet0/1
 description Client-LAN-Gateway
 ip address 10.1.1.1 255.255.255.0
 no shutdown

! Server DMZ Interface
interface GigabitEthernet0/2
 description Server-DMZ-Gateway
 ip address 10.1.2.1 255.255.255.0
 no shutdown

end
write memory
```

(Alternative for Linux Router node: `ip addr add 10.1.1.1/24 dev eth1 && ip addr add 10.1.2.1/24 dev eth2 && sysctl -w net.ipv4.ip_forward=1`).

Step 2: Configure Node IP Addressing & Default Gateways

1. **Client Node (`<initials>-H0`)**:

```
# Assign IP and default gateway pointing to R1 Gi0/1
ip addr add 10.1.1.10/24 dev eth0 2>/dev/null || ip addr add 10.1.1.10/24 dev wlan0
ip link set eth0 up 2>/dev/null || ip link set wlan0 up
ip route add default via 10.1.1.1
```

2. **DNS Server Node (`<initials>-DNS0`)**:

```
# Assign IP and default gateway pointing to R1 Gi0/2
ip addr add 10.1.2.53/24 dev eth0 2>/dev/null || ip addr add 10.1.2.53/24 dev wlan0
ip link set eth0 up 2>/dev/null || ip link set wlan0 up
ip route add default via 10.1.2.1
```

3. **Web Server Node (`<initials>-WEB0`)**:

```
# Assign IP and default gateway pointing to R1 Gi0/2
ip addr add 10.1.2.100/24 dev eth0 2>/dev/null || ip addr add 10.1.2.100/24 dev wlan0
ip link set eth0 up 2>/dev/null || ip link set wlan0 up
ip route add default via 10.1.2.1
```

4. **Verify Routing Reachability**: From `<initials>-H0` , ping across the router to both servers:

```
ping -c 2 10.1.2.53
ping -c 2 10.1.2.100
```

Step 3: Configure DNS Server (`<initials>-DNS0`)

On **<initials>-DNS0**, configure dnsmasq to resolve <lastname>.cs457.lab (e.g. haefner.cs457.lab) to 10.1.2.100:

```
# Start dnsmasq resolving <lastname>.cs457.lab -> 10.1.2.100
# Example for Prof. Haefner: dnsmasq -k --address=/haefner.cs457.lab/10.1.2.100 --listen-address=10.1.2.53 &
pkill dnsmasq 2>/dev/null
dnsmasq -k --address=/<lastname>.cs457.lab/10.1.2.100 --listen-address=10.1.2.53 &
```

On **<initials>-H0**, point local DNS resolution to <initials>-DNS0:

```
echo "nameserver 10.1.2.53" > /etc/resolv.conf

# Test DNS resolution across the router (e.g. nslookup haefner.cs457.lab 10.1.2.53)
nslookup <lastname>.cs457.lab 10.1.2.53
```

Step 4: Build Private PKI & Issue Server Certificate

On the Web Server node (**<initials>-WEB0**), create the private Root CA and sign the web server certificate using OpenSSL:

```
mkdir -p /etc/ssl/pki
cd /etc/ssl/pki

# 1. Generate Private Key & Self-Signed Root CA Certificate
openssl genrsa -out ca.key 2048
openssl req -x509 -new -nodes -key ca.key -sha256 -days 365 \
  -out ca.crt \
  -subj "/C=US/ST=Colorado/L=FortCollins/O=CS457-PKI/CN=CS457-Root-CA"

# 2. Generate Private Key & SAN Configuration for Web Server
openssl genrsa -out server.key 2048

# NOTE: Replace <lastname> below with your lower-case last name (e.g. haefner.cs457.lab)
cat << 'EOF' > san.cnf
[req]
distinguished_name = req_distinguished_name
req_extensions = v3_req
prompt = no

[req_distinguished_name]
C = US
ST = Colorado
L = FortCollins
O = CS457-Lab
CN = <lastname>.cs457.lab

[v3_req]
keyUsage = keyEncipherment, dataEncipherment
extendedKeyUsage = serverAuth
subjectAltName = @alt_names
```

```
[alt_names]
DNS.1 = <lastname>.cs457.lab
IP.1 = 10.1.2.100
EOF

# 3. Generate CSR & Sign Server Certificate with Root CA
openssl req -new -key server.key -out server.csr -config san.cnf
openssl x509 -req -in server.csr \
  -CA ca.crt -CAkey ca.key -CAcreateserial \
  -out server.crt -days 365 -sha256 -extfile san.cnf -extensions v3_req
```

Step 5: Configure HTTPS Web Server

On **`<initials>-WEB0`**, configure Nginx to serve HTTPS traffic over Port 443:

```
mkdir -p /var/www/html
cat << 'EOF' > /var/www/html/index.html
<!DOCTYPE html>
<html>
<head><title>CS457 Secure Web Server</title></head>
<body>
  <h1>Welcome to CS457 Routed TLS & PKI Web Server</h1>
  <p>This page was resolved via DNS (10.1.2.53) and encrypted via TLS (10.1.2.100)!</p>
</body>
</html>
EOF

# NOTE: Replace <lastname> with your lower-case last name (e.g. haefner.cs457.lab)
cat << 'EOF' > /etc/nginx/sites-available/default
server {
    listen 443 ssl;
    server_name <lastname>.cs457.lab;

    ssl_certificate /etc/ssl/pki/server.crt;
    ssl_certificate_key /etc/ssl/pki/server.key;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    location / {
        root /var/www/html;
        index index.html;
    }
}
EOF

nginx -t && nginx -s reload 2>/dev/null || service nginx restart
```

Step 6: Push Root of Trust (`ca.crt`) to Client & Verify

1. **Transfer Root CA Certificate to Client (`<initials>-H0`)**: On `<initials>-WEB0`, display `ca.crt`:

   ```
   cat /etc/ssl/pki/ca.crt
   ```

   On `<initials>-H0`, save `ca.crt`:

   ```
   cat << 'EOF' > /tmp/ca.crt
   [PASTE ca.crt CONTENT HERE]
   EOF
   ```

2. **Install Root CA into Client Trust Store**: On `<initials>-H0`:

   ```
   cp /tmp/ca.crt /usr/local/share/ca-certificates/cs457-root-ca.crt 2>/dev/null || cp
   /tmp/ca.crt /etc/ssl/certs/cs457-root-ca.crt
   update-ca-certificates 2>/dev/null
   ```

3. **Verify Untrusted vs. Trusted TLS Connection**: On `<initials>-H0`, test fetching the web page **without** specifying the CA certificate (fails due to untrusted CA):

   ```
   # Example: curl -v https://haefner.cs457.lab
   curl -v https://<lastname>.cs457.lab
   ```

   Now test **with** the Root of Trust passed explicitly:

   ```
   # Example: curl -v --cacert /tmp/ca.crt https://haefner.cs457.lab
   curl -v --cacert /tmp/ca.crt https://<lastname>.cs457.lab
   ```

   (Verify that output prints `SSL certificate verify ok.` and displays the encrypted HTML page!).

Step-by-Step Traffic Generation & Wireshark Capture:

1. **Start Packet Capture**: In CML GUI, right-click the Client (`<initials>-H0`) interface or Router (`<initials>-R1`) interface `Gi0/1`, select **Packet Capture**, and click **Start**.

2. **Generate Traffic**: On Client `<initials>-H0`, execute:

   ```
   curl -v --cacert /tmp/ca.crt https://<lastname>.cs457.lab
   ```

3. **Stop & Download Capture**: In CML GUI Packet Capture tab, click **Stop** and **Download** the `.pcap` file to open in Wireshark.

Extra Credit Challenge (+10 Points Bonus): Firefox GUI Browser Verification

To earn **10 bonus points**, add a **5th node** to your topology: a Linux Desktop GUI Node featuring the **Firefox web browser** (`<initials>-DESKTOP`).

Challenge Requirements:

1. **Network & DNS Configuration:** Configure the Desktop node on the Client LAN (`10.1.1.20/24`), set its default gateway to Router `<initials>-R1` (`10.1.1.1`), and set its DNS server to `<initials>-DNS0` (`10.1.2.53`).
2. **Push Root of Trust into Firefox:** Transfer and import your Root CA certificate (`ca.crt`) directly into Firefox's internal Certificate Store (*Settings* $\rightarrow$ *Privacy & Security* $\rightarrow$ *Certificates* $\rightarrow$ *View Certificates* $\rightarrow$ *Authorities* $\rightarrow$ *Import...*).
3. **Browser HTTPS Verification:** Open Firefox and navigate to `https://<lastname>.cs457.lab` (e.g. `https://haefner.cs457.lab`).
4. **Deliverable:** Include a screenshot of the Firefox browser displaying your web page with a **trusted secure padlock icon** (confirming a valid, trusted SSL/TLS connection with zero security warnings!).

> *[!NOTE] **Independent Configuration Challenge:** Step-by-step terminal commands for this extra credit task are intentionally withheld—students must apply what they have learned to configure the desktop IP/gateway/DNS settings and Firefox certificate manager independently!*

---

7. Wireshark Analysis and Questions

Open your packet capture in Wireshark. Filter by `dns || ssl || tls || tcp.port == 443` and answer the following questions:

Part A: DNS Resolution (UDP Port 53)

1. Locate the **DNS Query and Response** packets sent between Client (`10.1.1.10`) and DNS Server (`10.1.2.53`).
   - What is the packet number for the DNS Query for `<lastname>.cs457.lab` (e.g. `haefner.cs457.lab`)?
   - What IP address is returned in the DNS Answer?
   - What transport protocol and port number does DNS use for standard domain queries?

Part B: Inter-Subnet TCP Connection

2. Locate the **TCP 3-Way Handshake** between Client (`10.1.1.10`) and Web Server (`10.1.2.100`).
   - What are the packet numbers for SYN, SYN-ACK, and ACK?
   - Explain how packets travel across subnets between `10.1.1.0/24` and `10.1.2.0/24` via default gateway `10.1.1.1` (`R1`).

Part C: PKI & Server Certificate Inspection

3. Locate the **TLS Certificate** message sent from Server to Client.
   - What is the **Common Name (CN)** of the Subject in your server certificate (e.g. `haefner.cs457.lab`)?
   - What is the **Common Name (CN)** of the Issuer CA in the server certificate (`CS457-Root-CA`)?
   - What digital signature algorithm was used by your private CA to sign the certificate?
   - What are the first four hexadecimal digits of the server's public key modulus?

Part D: TLS Handshake & Cipher Suite Negotiation

4. Inspect the **TLS Client Hello** message:
   - What version of TLS is declared by the client?
   - How many cipher suites were offered by the client?
   - What are the first two hexadecimal digits of the `Random Bytes` field? What is the purpose of this random nonce?
5. Inspect the **TLS Server Hello** message:
   - Which specific cipher suite was chosen by the server?

- What symmetric encryption algorithm and key length does this cipher suite use?

Part E: Root of Trust Verification & Application Data

6. Locate the **Application Data** packets following the handshake:
   - Why are the HTTP GET request and HTTP response payloads encrypted and unreadable in Wireshark?
   - Explain how installing `ca.crt` on the client enabled `curl` to verify the server's identity and establish trust without security warnings.

---

8. Troubleshooting

Issue Encountered: [Describe any issues with routing, DNS resolution, OpenSSL, Nginx, or curl verification]
Discovery Method: [How did you identify the issue?]
Resolution: [How did you fix it?]

(If curl reported `Could not resolve host`, check `/etc/resolv.conf` and `dnsmasq` status on `<initials>-DNS0`).

---

9. Conclusion & Analysis

Summarize your findings. Detail how the complete secure web architecture integrates inter-subnet IP routing, DNS name resolution (`<lastname>.cs457.lab`), PKI certificate validation, and TLS session encryption to deliver end-to-end security.

---

10. What to Hand In

- A single PDF report containing:
  - Your written answers to all 6 Wireshark analysis questions in Section 7.
  - Screenshot 1: Active CML topology showing running Client (`<initials>-H0`), Router (`<initials>-R1`), DNS (`<initials>-DNS0`), and Web Server (`<initials>-WEB0`).
  - Screenshot 2: DNS Server console showing `dnsmasq` running and successful `nslookup <lastname>.cs457.lab` (e.g. `haefner.cs457.lab`) on Client.
  - Screenshot 3: Web Server console showing successful creation of `ca.crt`, `server.key`, and `server.crt` using OpenSSL.
  - Screenshot 4: Client console showing successful `curl -v --cacert /tmp/ca.crt https://<lastname>.cs457.lab` output confirming `SSL certificate verify ok.`.
  - Screenshot 5: Wireshark capture window showing DNS query/response, TCP 3-way handshake, and TLS Handshake / Application Data.
  - **Optional Extra Credit Screenshot (+10 Points):** Screenshot of Firefox browser on Desktop node `<initials>-DESKTOP` displaying `https://<lastname>.cs457.lab` with a fully trusted secure padlock icon.